



Universidade de Brasília  
CIC - UNB

**Sistema CAMAAR**

# **Modelo Relacional e Investigação Técnica das Views**

**Integrantes:**

|                                    |           |
|------------------------------------|-----------|
| Ana Luísa Reis Nascente            | 211045688 |
| Gabriel de Sousa                   | 211056000 |
| Maria Luiza Rodrigues de Sousa     | 221007223 |
| Arthur Henrique Aprigio dos Santos | 232000481 |

Brasília DF  
19/05/2026

# Sumário

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| <b>1</b>  | <b>Resumo</b>                                    | <b>2</b>  |
| <b>2</b>  | <b>Introdução</b>                                | <b>2</b>  |
| <b>3</b>  | <b>Objetivos</b>                                 | <b>2</b>  |
| <b>4</b>  | <b>Visão Geral do Sistema</b>                    | <b>2</b>  |
| <b>5</b>  | <b>Modelo Relacional</b>                         | <b>3</b>  |
| 5.1       | Modelo Relacional UML . . . . .                  | 3         |
| 5.2       | Dicionário de Dados . . . . .                    | 3         |
| 5.3       | Entidades Principais . . . . .                   | 4         |
| 5.3.1     | User . . . . .                                   | 4         |
| 5.3.2     | Template . . . . .                               | 4         |
| 5.3.3     | Question . . . . .                               | 5         |
| 5.3.4     | FormResponse . . . . .                           | 5         |
| 5.4       | Cardinalidades e Regras de Negócio . . . . .     | 5         |
| 5.5       | Observação Técnica sobre o Diagrama . . . . .    | 6         |
| <b>6</b>  | <b>Investigação Técnica das Views</b>            | <b>6</b>  |
| 6.1       | Fluxo de Autenticação . . . . .                  | 6         |
| 6.1.1     | Tela de Login . . . . .                          | 6         |
| 6.1.2     | Tela de Alteração de Senha . . . . .             | 6         |
| 6.2       | Layout Principal . . . . .                       | 6         |
| 6.3       | Módulo de Avaliações . . . . .                   | 7         |
| 6.3.1     | Tela Inicial de Avaliações . . . . .             | 7         |
| 6.3.2     | Tela de Resposta da Avaliação . . . . .          | 7         |
| 6.4       | Módulo de Gerenciamento . . . . .                | 7         |
| 6.4.1     | Tela Inicial de Gerenciamento . . . . .          | 7         |
| 6.4.2     | Fluxo de Importação de Dados . . . . .           | 8         |
| 6.5       | Fluxo de Templates . . . . .                     | 8         |
| 6.6       | Fluxo de Resultados . . . . .                    | 8         |
| <b>7</b>  | <b>Estrutura Sugerida em Ruby on Rails</b>       | <b>9</b>  |
| 7.1       | Views e Partial . . . . .                        | 9         |
| 7.2       | Controllers . . . . .                            | 9         |
| 7.3       | Entidades Complementares Identificadas . . . . . | 10        |
| <b>8</b>  | <b>Principais Fluxos de Uso</b>                  | <b>10</b> |
| 8.1       | Fluxo do Avaliador . . . . .                     | 10        |
| 8.2       | Fluxo do Administrador . . . . .                 | 11        |
| <b>9</b>  | <b>Boas Práticas de Implementação</b>            | <b>11</b> |
| <b>10</b> | <b>Considerações Finais</b>                      | <b>11</b> |

# 1 Resumo

Este relatório apresenta uma análise técnica da aplicação **CAMAAR**, com foco em dois aspectos centrais do projeto: o modelo relacional utilizado para organizar usuários, templates, perguntas e respostas; e a investigação das views necessárias para implementar os fluxos de autenticação, avaliações, gerenciamento, templates, importação de dados e resultados.

O documento também descreve as cardinalidades do modelo entidade-relacionamento, as responsabilidades de cada tabela, a estrutura esperada para uma aplicação em Ruby on Rails e as decisões de organização visual sugeridas a partir dos protótipos do sistema.

## 2 Introdução

A aplicação **CAMAAR** é um sistema web voltado ao gerenciamento e à avaliação de atividades acadêmicas. A proposta do sistema é permitir que usuários autenticados respondam avaliações, enquanto usuários com permissões administrativas possam importar dados, criar templates de avaliação, enviar formulários e acompanhar resultados.

Em termos de arquitetura, o sistema pode ser compreendido como uma aplicação administrativa organizada em dois grandes módulos:

- **Avaliações:** módulo usado pelos respondentes para visualizar avaliações disponíveis e registrar respostas.
- **Gerenciamento:** módulo usado por administradores para importar informações, gerenciar templates, enviar formulários e consultar resultados.

Este relatório consolida a modelagem relacional e a investigação das views com o objetivo de orientar as próximas etapas de desenvolvimento em Ruby on Rails.

## 3 Objetivos

Os objetivos deste relatório são:

- documentar as entidades principais do modelo relacional da aplicação;
- explicar os atributos, chaves primárias e chaves estrangeiras de cada tabela;
- registrar as regras de negócio e as cardinalidades representadas no diagrama;
- mapear as principais views e partials esperadas para a implementação;
- indicar os controllers, dados e fluxos necessários para as telas do sistema;
- organizar a documentação em formato adequado para um relatório técnico de computação.

## 4 Visão Geral do Sistema

O CAMAAR possui uma estrutura típica de sistema acadêmico com autenticação, controle de permissões e separação entre área de resposta e área administrativa. Após o login, o usuário é direcionado para telas internas com navegação lateral, barra superior, cards de conteúdo e formulários.

O sistema deve oferecer suporte a dois perfis principais:

- **Usuário respondente:** acessa avaliações disponíveis, seleciona uma disciplina ou atividade e envia respostas.
- **Usuário administrador:** acessa funcionalidades de gerenciamento, cria templates, importa disciplinas ou turmas, envia formulários e visualiza resultados.

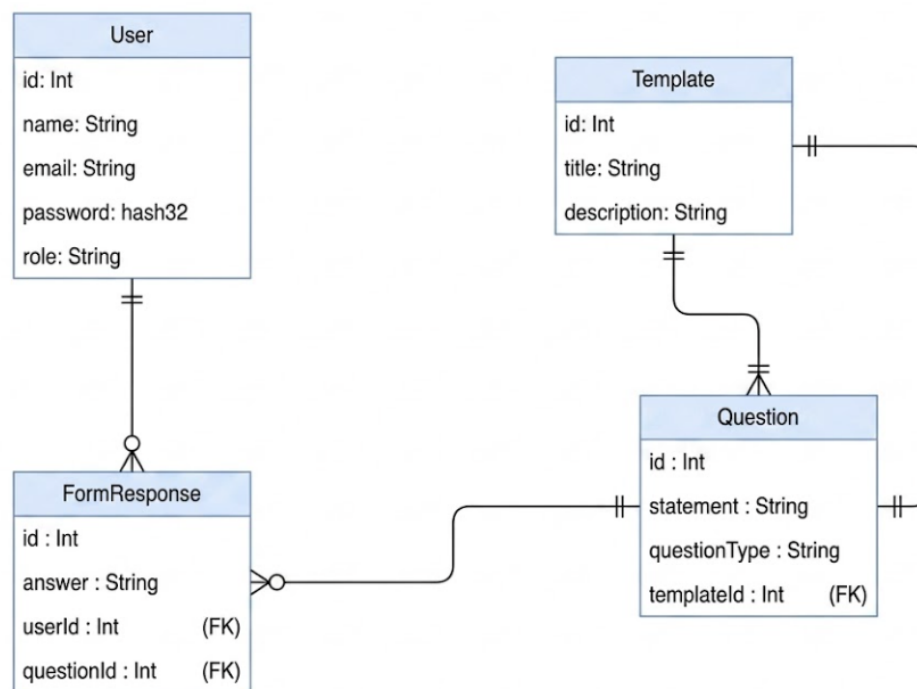
Essa separação de responsabilidades influencia diretamente tanto o modelo relacional quanto a organização das views. No modelo de dados, as respostas precisam estar vinculadas ao usuário e à pergunta respondida. Na camada de apresentação, as telas devem separar claramente os fluxos de avaliação e os fluxos administrativos.

## 5 Modelo Relacional

O modelo relacional descreve a estrutura mínima necessária para representar formulários, perguntas e respostas no sistema. Ele é composto pelas entidades **User**, **Template**, **Question** e **FormResponse**. Essas entidades permitem cadastrar usuários, criar modelos de formulários, associar perguntas a cada template e registrar respostas individuais.

### 5.1 Modelo Relacional UML

A Figura 1 apresenta o modelo relacional UML da aplicação **CAMAAR**.



**Figura 1:** Modelo Relacional UML da aplicação CAMAAR.

### 5.2 Dicionário de Dados

O dicionário de dados descreve a responsabilidade de cada tabela e os principais atributos representados no modelo.

| Tabela              | Atributos                                         | Descrição                                                                                                                                                                                                                                              |
|---------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>User</b>         | id (PK), name, email, password, role              | Armazena os dados cadastrais dos usuários que interagem com o sistema. O campo <b>role</b> indica o nível de permissão, como administrador ou usuário comum. O campo de senha deve representar uma senha criptografada, nunca uma senha em texto puro. |
| <b>Template</b>     | id (PK), title, description                       | Representa o modelo geral de um formulário ou avaliação. Define o título, a descrição e a estrutura base que será composta por perguntas.                                                                                                              |
| <b>Question</b>     | id (PK), statement, questionType, templateId (FK) | Armazena as perguntas pertencentes a um template. O campo <b>questionType</b> define o formato de resposta, como texto, múltipla escolha, escala ou checkbox.                                                                                          |
| <b>FormResponse</b> | id (PK), answer, userId (FK), questionId (FK)     | Registra a resposta enviada por um usuário para uma pergunta específica. A tabela funciona como ponto de ligação entre respondente e pergunta.                                                                                                         |

## 5.3 Entidades Principais

### 5.3.1 User

A entidade **User** representa as pessoas que acessam o sistema. Ela concentra dados de autenticação e autorização, como nome, e-mail, senha criptografada e papel do usuário.

- **id**: identificador único do usuário.
- **name**: nome completo do usuário.
- **email**: e-mail usado para identificação e login.
- **password**: representação criptográfica da senha.
- **role**: papel do usuário no sistema.

Um usuário pode responder várias perguntas ao longo do sistema, mas cada resposta registrada deve estar associada a exatamente um usuário.

### 5.3.2 Template

A entidade **Template** representa o esqueleto de uma avaliação. Ela agrupa perguntas e permite que administradores reutilizem estruturas de formulários em diferentes disciplinas, turmas ou semestres.

- **id**: identificador único do template.
- **title**: título do formulário ou avaliação.

- **description**: descrição, instruções ou objetivo do template.

Um template possui uma ou muitas perguntas. Essa relação permite que o formulário seja montado de forma modular.

### 5.3.3 Question

A entidade **Question** registra cada pergunta pertencente a um template. Cada pergunta possui um enunciado e um tipo, o que permite renderizar campos diferentes na interface.

- **id**: identificador único da pergunta.
- **statement**: enunciado da pergunta.
- **questionType**: tipo da pergunta e da resposta esperada.
- **templateId**: chave estrangeira associada ao template.

Cada pergunta pertence obrigatoriamente a um template e pode receber nenhuma, uma ou muitas respostas.

### 5.3.4 FormResponse

A entidade **FormResponse** registra as respostas submetidas pelos usuários. Ela associa uma resposta a um usuário e a uma pergunta, preservando a rastreabilidade do preenchimento.

- **id**: identificador único da resposta.
- **answer**: conteúdo informado pelo usuário.
- **userId**: chave estrangeira associada ao usuário respondente.
- **questionId**: chave estrangeira associada à pergunta respondida.

Cada resposta pertence obrigatoriamente a um usuário e a uma pergunta específica.

## 5.4 Cardinalidades e Regras de Negócio

As cardinalidades indicam como as entidades se relacionam e quais restrições devem ser respeitadas pela aplicação e pelo banco de dados.

- **User para FormResponse (1:N)**: um usuário pode responder zero ou muitas perguntas. Cada resposta deve pertencer a exatamente um usuário.
- **Question para FormResponse (1:N)**: uma pergunta pode receber zero ou muitas respostas. Cada resposta deve estar vinculada a exatamente uma pergunta.
- **Template para Question (1:N)**: um template é composto por uma ou muitas perguntas. Cada pergunta deve pertencer a exatamente um template.

Do ponto de vista de integridade referencial, as chaves estrangeiras **templateId**, **userId** e **questionId** devem ser validadas para evitar respostas sem usuário, respostas sem pergunta ou perguntas sem template.

## 5.5 Observação Técnica sobre o Diagrama

No diagrama há uma ligação duplicada entre **Template** e **Question** com aparência de cardinalidade 1:1. Como a estrutura de chaves do modelo indica que `templateId` está em **Question**, a relação correta é 1:N, isto é, um template possui muitas perguntas. Assim, a segunda linha deve ser interpretada como artefato visual da ferramenta de modelagem ou erro gráfico, e não como uma regra de negócio adicional.

## 6 Investigação Técnica das Views

A análise dos protótipos indica que a interface do CAMAAR deve seguir uma organização administrativa simples, com menu lateral, barra superior e páginas internas renderizadas conforme o módulo selecionado.

Em Ruby on Rails, essa estrutura pode ser implementada com um layout principal compartilhado e partials reutilizáveis para elementos comuns, como navegação, mensagens de sistema, cards, tabelas e campos de formulário.

### 6.1 Fluxo de Autenticação

#### 6.1.1 Tela de Login

A tela de login permite que o usuário acesse o sistema informando e-mail e senha. Visualmente, o protótipo sugere um formulário central ou lateral acompanhado de um painel institucional.

- **View provável:** `app/views/sessions/new.html.erb`.
- **Alternativa com Devise:** `app/views/devise/sessions/new.html.erb`.
- **Elementos:** campo de e-mail, campo de senha, botão de entrada e mensagem institucional.
- **Fluxo:** preencher credenciais, submeter o formulário, validar a autenticação e redirecionar para o módulo de avaliações.

#### 6.1.2 Tela de Alteração de Senha

A tela de alteração de senha permite redefinir credenciais de acesso.

- **View provável:** `app/views/passwords/edit.html.erb`.
- **Alternativa com Devise:** `app/views/devise/passwords/edit.html.erb`.
- **Elementos:** campo de nova senha, campo de confirmação e botão para alterar senha.
- **Fluxo:** informar nova senha, confirmar senha, validar campos, atualizar credenciais e redirecionar para o sistema.

### 6.2 Layout Principal

O layout principal deve concentrar a estrutura comum das páginas autenticadas. Essa abordagem evita duplicação de código e facilita manutenção visual.

- `app/views/layouts/application.html.erb`: layout base da aplicação.
- `app/views/shared/_navbar.html.erb`: barra superior com nome da página, pesquisa, avatar e ação de logout.
- `app/views/shared/_sidebar.html.erb`: menu lateral com navegação entre Avaliações e Gerenciamento.
- `app/views/shared/_flash.html.erb`: mensagens de erro, sucesso e aviso.

A barra superior deve apresentar elementos de contexto da página atual, enquanto o menu lateral deve destacar o item ativo e permitir navegação rápida entre os módulos principais.

## 6.3 Módulo de Avaliações

### 6.3.1 Tela Inicial de Avaliações

A tela inicial de avaliações deve listar as avaliações disponíveis para o usuário autenticado. A representação por cards facilita a identificação de disciplina, semestre e professor.

- **View:** `app/views/evaluations/index.html.erb`.
- **Partial:** `app/views/evaluations/_evaluation_card.html.erb`.
- **Dados necessários:** `@evaluations`.
- **Campos esperados:** nome da matéria, semestre e nome do professor.

O fluxo esperado é carregar as avaliações disponíveis para o usuário, apresentar os cards e abrir o formulário correspondente quando uma avaliação for selecionada.

### 6.3.2 Tela de Resposta da Avaliação

A tela de resposta da avaliação deve renderizar dinamicamente as perguntas do template vinculado à avaliação. Como o sistema pode possuir perguntas objetivas, discursivas e escalas de avaliação, a renderização deve ser modular.

- **View principal:** `app/views/evaluations/show.html.erb`.
- **Alternativa:** `app/views/evaluation_responses/new.html.erb`.
- **Partials:** `app/views/evaluations/_question.html.erb`, `app/views/evaluations/_multiple_choice_question.html.erb` e `app/views/evaluations/_text_question.html.erb`.
- **Dados necessários:** `@evaluation`, `@questions` e `@response`.

O fluxo esperado é abrir a avaliação, carregar perguntas, renderizar os campos adequados para cada tipo, salvar as respostas e finalizar a avaliação.

## 6.4 Módulo de Gerenciamento

### 6.4.1 Tela Inicial de Gerenciamento

A tela inicial de gerenciamento centraliza as funcionalidades administrativas. Ela deve apresentar acesso rápido para importação de dados, edição de templates, envio de formulários e consulta de resultados.

- **View:** `app/views/management/index.html.erb`.
- **Ações principais:** importar dados, editar templates, enviar formulários e visualizar resultados.
- **Perfil esperado:** usuário administrador ou usuário com permissão equivalente.

#### 6.4.2 Fluxo de Importação de Dados

O fluxo de importação deve permitir selecionar disciplinas ou turmas que serão usadas na criação e no envio de avaliações.

- **View:** `app/views/management/imports/new.html.erb`.
- **Partial:** `app/views/management/imports/_table.html.erb`.
- **Elementos:** lista de turmas, tabela, checkboxes e botão de envio.
- **Dados necessários:** `@subjects` e `@semesters`.

O fluxo esperado é selecionar disciplinas, confirmar a ação e persistir os dados importados para uso posterior.

### 6.5 Fluxo de Templates

O fluxo de templates é responsável pela criação, edição e exclusão de modelos de avaliação.

- **View de listagem:** `app/views/templates/index.html.erb`.
- **Views de formulário:** `app/views/templates/new.html.erb` e `app/views/templates/edit.html.erb`.
- **Form reutilizável:** `app/views/templates/_form.html.erb`.
- **Partials:** `app/views/templates/_template_card.html.erb` e `app/views/templates/_question_fields.html.erb`.
- **Dados necessários:** `@templates`, `@template` e `@question_types`.

Na criação ou edição de templates, a interface deve permitir informar o nome do template, selecionar semestre, cadastrar perguntas, definir tipos de pergunta, adicionar opções quando necessário e salvar a estrutura final.

### 6.6 Fluxo de Resultados

O fluxo de resultados deve permitir que administradores acompanhem avaliações respondidas e métricas associadas.

- **View de listagem:** `app/views/results/index.html.erb`.
- **View detalhada:** `app/views/results/show.html.erb`.
- **Partial:** `app/views/results/_result_card.html.erb`.
- **Elementos:** cards de matérias, professor, semestre, quantidade de respostas, médias, gráficos, comentários discursivos e filtros.

O fluxo esperado é acessar resultados, selecionar uma matéria ou avaliação e visualizar estatísticas detalhadas.

## 7 Estrutura Sugerida em Ruby on Rails

### 7.1 Views e Partial

A estrutura sugerida para as views é:

- `app/views/layouts/application.html.erb`
- `app/views/shared/_navbar.html.erb`
- `app/views/shared/_sidebar.html.erb`
- `app/views/shared/_flash.html.erb`
- `app/views/sessions/new.html.erb`
- `app/views/passwords/edit.html.erb`
- `app/views/evaluations/index.html.erb`
- `app/views/evaluations/show.html.erb`
- `app/views/evaluations/_evaluation_card.html.erb`
- `app/views/evaluations/_question.html.erb`
- `app/views/evaluations/_multiple_choice_question.html.erb`
- `app/views/evaluations/_text_question.html.erb`
- `app/views/management/index.html.erb`
- `app/views/management/imports/new.html.erb`
- `app/views/management/imports/_table.html.erb`
- `app/views/templates/index.html.erb`
- `app/views/templates/new.html.erb`
- `app/views/templates/edit.html.erb`
- `app/views/templates/_form.html.erb`
- `app/views/templates/_template_card.html.erb`
- `app/views/templates/_question_fields.html.erb`
- `app/views/results/index.html.erb`
- `app/views/results/show.html.erb`
- `app/views/results/_result_card.html.erb`

### 7.2 Controllers

Com base nos fluxos identificados, os controllers sugeridos são:

- **SessionsController**: autenticação e encerramento de sessão.
- **PasswordsController**: alteração ou recuperação de senha.
- **EvaluationsController**: listagem e exibição de avaliações.

- **EvaluationResponsesController**: criação e submissão de respostas.
- **ManagementController**: página inicial da área administrativa.
- **ImportsController**: importação ou seleção de disciplinas e turmas.
- **TemplatesController**: gerenciamento de templates e perguntas.
- **ResultsController**: visualização de métricas e resultados.

Essa separação mantém cada controller responsável por um conjunto claro de ações, facilitando testes, manutenção e evolução do sistema.

## 7.3 Entidades Complementares Identificadas

Além das quatro entidades presentes no modelo relacional simplificado, a análise das views sugere que versões futuras do sistema podem exigir entidades complementares:

- **Subject**: disciplina avaliada.
- **Teacher**: professor associado à disciplina ou turma.
- **Semester**: semestre de oferta.
- **Evaluation**: instância de avaliação enviada aos usuários.
- **EvaluationTemplate**: associação entre avaliação e template.
- **QuestionOption**: opções de perguntas objetivas.
- **Answer**: resposta individual em uma submissão.
- **ImportedSubject**: disciplina importada para uso administrativo.

Essas entidades não são obrigatórias para o modelo simplificado atual, mas ajudam a representar melhor os fluxos de importação, avaliação por disciplina e análise de resultados.

# 8 Principais Fluxos de Uso

## 8.1 Fluxo do Avaliador

O fluxo do avaliador é composto pelas seguintes etapas:

1. realizar login;
2. visualizar a lista de avaliações disponíveis;
3. selecionar uma matéria ou avaliação;
4. responder perguntas objetivas e discursivas;
5. enviar a avaliação;
6. receber confirmação de envio.

## 8.2 Fluxo do Administrador

O fluxo administrativo pode seguir três caminhos principais:

1. realizar login, acessar gerenciamento, importar dados, selecionar turmas e enviar as informações;
2. realizar login, acessar gerenciamento, abrir templates, criar ou editar perguntas e salvar o template;
3. realizar login, acessar gerenciamento, abrir resultados, selecionar uma matéria e visualizar estatísticas.

## 9 Boas Práticas de Implementação

Para manter a aplicação organizada e compatível com boas práticas de desenvolvimento web em Ruby on Rails, recomenda-se:

- reutilizar partials para cards, tabelas, perguntas e componentes comuns;
- manter regras de negócio nos models e services, evitando excesso de lógica nas views;
- usar validações nos models para garantir integridade de templates, perguntas e respostas;
- aplicar controle de autorização por papel de usuário, principalmente nas telas de gerenciamento;
- renderizar perguntas de forma dinâmica conforme `questionType`;
- exibir mensagens de sucesso e erro por meio de partial de flash;
- manter nomenclatura consistente entre models, controllers, rotas e views;
- preparar a interface para responsividade e acessibilidade básica.

## 10 Considerações Finais

O modelo relacional apresentado oferece uma base simples e coerente para registrar usuários, templates, perguntas e respostas. A separação entre **Template**, **Question** e **FormResponse** favorece reutilização de formulários, controle das respostas e expansão futura da aplicação.

A investigação técnica das views mostra que o CAMAAR deve ser organizado como uma aplicação Rails modular, com layout administrativo compartilhado, partials reutilizáveis e controllers separados por responsabilidade. Essa estrutura permite evoluir o sistema de forma incremental, mantendo clareza entre os fluxos de avaliação e gerenciamento.

Com a documentação do modelo relacional e das views, o projeto passa a ter uma referência técnica mais completa para orientar a implementação das próximas sprints.